Chapter 5

CONSTRUCTIVE MODELS

Decomposition models discussed in the preceding chapter represent solids as a collection of basic elements, combined with a "gluing" operation. In contrast, the *constructive models* to be discussed in this chapter use much more powerful combination operations.

5.1 HALF-SPACE MODELS

All constructive models consider solids as point sets of E^3. Their basic idea is to start from some sufficiently simple point sets that can be represented directly, and model other point sets in terms of very general combinations of the simple sets. So-called *half-space* models apply this approach in a direct fashion.

5.1.1 Half-Spaces

Every point set A can be thought of as having a *characteristic function* $g_A(X) : X \rightarrow \{0, 1\}$ which tells whether a point X is considered to be a member of A or not. In other words, the function g_A must satisfy

$$g_A(X) = 1 \quad \Rightarrow X \in A$$

$$g_A(X) = 0 \quad \Rightarrow X \notin A$$

For very general point sets characteristic functions do not offer much help, because their representation would be as hard as the representation

of the sets themselves. However, for an interesting class of point sets g_A can be represented in terms of a real-valued analytic function f of x, y, and z defined everywhere in E^3. The restriction to analytic functions excludes certain "pathological" objects [94] as explained in Chapter 3. All points $X = (x\ y\ z)$ such that $f(X) \geq 0$ are considered to belong to the point set, while $f(X) < 0$ defines its complement.

Because $f(X) = 0$ divides the whole space into two subsets, point sets defined by $f(X) \geq 0$ and $f(X) \leq 0$ are referred to as *half-spaces*. For instance, functions

$$ax + by + cz + d \geq 0$$

$$x^2 + y^2 - r^2 \leq 0$$

define useful point sets, namely the *planar half-space* that consists of all points on or in the positive side of the plane $ax + by + cz + d = 0$, and the *cylindrical half-space* that consists of all points or or inside an infinite cylinder whose axis = z-axis and radius = r.

Other half-spaces of interest include the remaining *natural quadratic surfaces* such as spheres and cones, and certain higher-order surfaces such as tori. Observe that this collection includes both unbounded half-spaces (such as the infinite cylinder above) and bounded half-spaces (such as the sphere $x^2 + y^2 + z^2 - r^2 \leq 0$).

Not all surfaces of interest are half-spaces. For instance, the various *surface patches* widely used in surface models are not half-spaces, because they do not divide the space into distinct subsets. This means that the modeling space of a half-space modeler cannot easily be extended to cover also objects bounded by surface patches.

5.1.2 Boolean Set Operations

Half-spaces form the basic modeling primitives of half-space models. As half-spaces are point sets, the natural modeling procedures for half-space models are the *Boolean set operations* union ($\cup$), intersection ($\cap$) and set difference ($\setminus$).

Hence, half-space models are constructed by combining instances of half-spaces with Boolean set operations. For instance, to describe a finite cylinder C of length h, we need one cylindrical half-space and two planar half-spaces, combined together with the set operation "$\cap$":

$$\begin{aligned} H_1 &: \ x^2 + y^2 - r^2 \leq 0 \\ H_2 &: \ z \geq 0 \\ H_3 &: \ z - h \leq 0 \\ C &= \ H_1 \cap H_2 \cap H_3 \end{aligned}$$

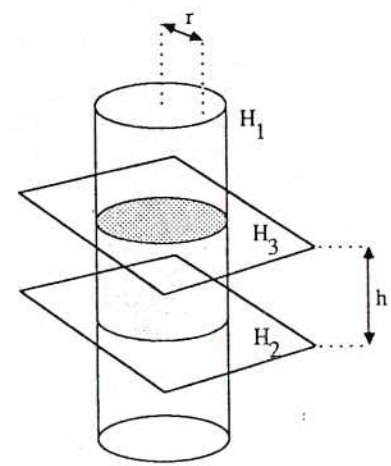

Figure 5.1 Half-space model of a finite cylinder.

This construction is illustrated in Figure 5.1.

Hence, the modeling space M of a half-space modeler is the class of *Boolean combinations* of the available half-spaces.

5.1.3 Representation of Half-Space Models

According to the above, the representation of a half-space model breaks into two parts:

1. *Representation of half-spaces*: The most common approach is to represent half-spaces as instances of half-space types, described with a half-spaces type code and a list of size parameters in some convenient coordinate system, and a transformation matrix that gives the actual location and orientation of the half-space.

   As an alternative to this approach, all quadratic half-spaces can be brought into the same form by writing their defining function $f(X) = f(x, y, z)$ as

$$f(x, y, z) = [x\ y\ z\ 1] \begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ A_{21} & A_{22} & A_{23} & A_{24} \\ A_{31} & A_{32} & A_{33} & A_{34} \\ A_{41} & A_{42} & A_{43} & A_{44} \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

and storing the 16 coefficients A_{ij}, or the 10 coefficients derived by evaluating the matrix products. While this general form offers advantages, e.g., in ray casting (to be explained in Section 5.2.2 on page 92), the surface type coding is more commonly used because of the conveniences it offers, e.g., in the calculation of surface-surface intersections.

2. *Representation of Boolean combinations of half-spaces*: One possibility is offered by the fact that every Boolean expression may be brought into the "sum-of-products" form. TIPS [91], a solid modeler based on the pure half-space approach, follows this approach by representing a solid as the union of the intersections of the individual half-spaces. Hence, a solid S is expressed in TIPS as

$$S = \bigcup_i \bigcap_j H_{ij},$$

where H_{ij} denote the individual half-spaces of S. Other approaches to representing combinations of half-spaces are discussed later in this chapter.

5.1.4 Properties of Half-Space Models

Properties of pure half-space modelers can be described in the already familiar framework.

Expressive power: The modeling space of a half-space modeler is determined by the selection of half-spaces available, and of the generality of the operations available for combining them. Typically, modelers of this class include planar and quadratic half-spaces (such as spherical, cylindrical, and conical surfaces), sometimes even tori. Note, however, that patch surfaces are *not* half-spaces and cannot hence be included into the modeling space of half-space modelers.

Validity: Half-spaces are (usually) infinite point sets. Even a combination of them may be infinite, and hence not all combinations are valid solids (if finiteness is a required characteristic of a "solid" point set). TIPS circumvents this difficulty by enforcing the definition of a "box of interest" as a part of each solid description; only the part of space within the box is considered to form the solid.

Unambiguity and uniqueness: Each valid combination of half-spaces determines a solid. Hence half-space models are unambiguous. Half-space representations are not unique.

Description languages: Instantiation and combination of half-spaces leads easily to text-oriented, relatively simple solid descriptions. With some effort, it is also possible to create a drafting-type graphical user interface.

Conciseness: Half-space models are relatively concise: a few hundred half-spaces are usually sufficient to model realistic parts adequately (within the limitations of the modeling space).

Closure of operations: Any combination of two half-space models with a Boolean set operation defines a new valid model; hence these operations are closed.

Computational ease and applicability: The natural algorithms for half-space modeling are based on so-called *set membership classification*. In particular, the *ray casting approach* to be explained in the context of CSG models in Section 5.2.2 is a simple and general way of attacking the computational problems of half-space models. These families of algorithms are discussed in the next section in connection with the related CSG models.

5.2 CONSTRUCTIVE SOLID GEOMETRY

Pure half-space models offer a mathematically rigorous, easily understandable approach to solid modeling. For human users, however, it is easier to operate with bounded primitives instead of the unbounded half-spaces. Observe also that combinations of half-spaces may be unbounded point sets that do not quite match our notion of a valid solid. To avoid the generation of unbounded sets, the so-called *Constructive Solid Geometry* (CSG) approach to solid modeling uses only bounded point sets as its primitives.

5.2.1 Representation of CSG Models

CSG adopts the "building block" approach to solid modeling in its pure form. The user of a CSG modeler operates only on parameterized instances of *solid primitives* and Boolean set operations on them. Each primitive, in turn, is defined as a combination of half-spaces; the user has no direct access to individual half-spaces, however. For instance, the user may create planar and cylindrical half-spaces by means of a cylinder primitive, but he cannot directly manipulate them. Hence the mathematical modeling space of CSG models is exactly the same as that of pure half-space models, except that only finite point sets are included.

The most natural way to represent a CSG model is the so-called *CSG tree* that can be defined as follows:

```
<CSG tree>  ::=  <primitive> |
                 <CSG tree> <set operation> <CSG tree> |
                 <CSG tree> <rigid motion>
```

In the above, <*primitive*> is an instance of a solid primitive, represented through a primitive type identifier and a sequence of dimension

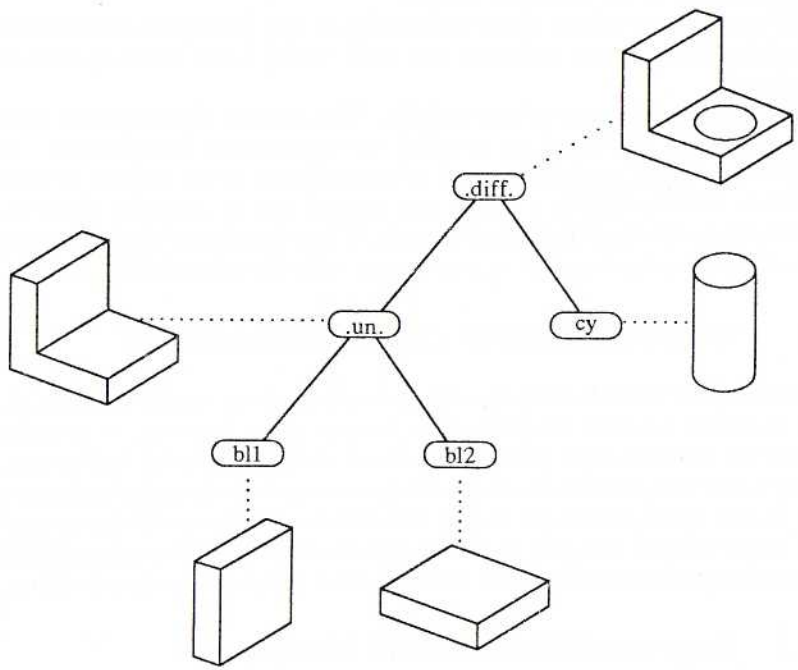

Figure 5.2 A CSG tree.

parameters. <*rigid motion*> is either a translation or a rotation, and <*set operation*> is one of $\cup$, $\cap$, and $\backslash$.

Hence, primitives are represented in the leaves of the CSG tree, while interior nodes are marked with either a Boolean set operation or with a rigid motion (see Figure 5.2). In this fashion, set operations and motions are interpreted to operate on CSG trees. This naturally leads to the creation CSG trees that model subassemblies of a design, which after appropriate transformations are used several times in the CSG tree representing the assembled design. In this case the binary tree actually becomes a *directed acyclic graph*.

Each primitive is chosen so as to define a bounded point set of E^3.

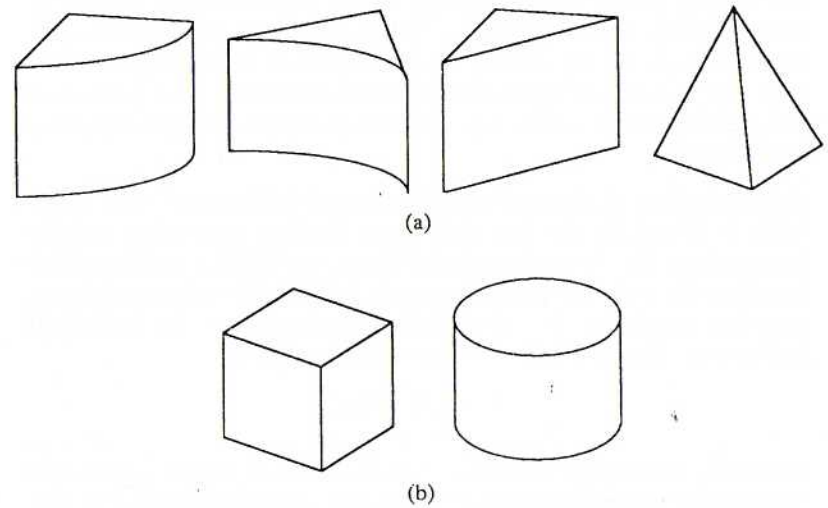

Figure 5.3 Two collections of CSG primitives.

As the set operations available cannot destroy boundedness, CSG models are guaranteed to define bounded sets. The ease of use of a CSG modeler depends much on the collection of primitives available. For instance, Figure 5.3 depicts two collections of CSG primitives. Note how collection (a) includes various wedges to aid the rounding of edges often needed in mechanical parts. Observe that all primitives of the figure can be expressed as a Boolean combination of simple half-spaces (in this case, planes and cylinders).

The actual domain of a CSG modeler depends on the variety of half-spaces available in its primitives, on the available rigid motions, and on the available set operations. Note that the two collections in Figure 5.3 have the same domain despite different primitives, because the underlying collection of half-spaces available is the same in both cases. (Actually, a CSG modeler with just one cylinder primitive has exactly the same domain.)

Regular Set Operations

Some combinations of CSG primitives (or half-spaces) do not quite satisfy our notion of "solidity." Consider, for instance, the case depicted in Figure 5.4. According to the ordinary definition of point set operations,

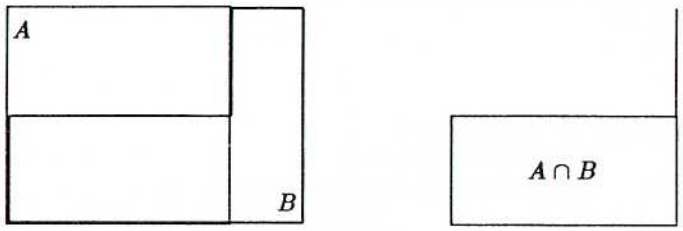

Figure 5.4 A nonregular set operation.

the intersection of the two objects consists of a rectangular object plus a "dangling" line segment (b). This effect is undesirable, for instance, if set operations are used to test the validity of an assembly by intersecting all component pairs and checking whether the result is an empty object. In that case, we would prefer that the intersection of merely touching components is empty.

In light of the point-set characterization of solidity, the problem of the resulting object is that it is not regular: the "dangling line" violates the Definition 3.3 of regularity on page 33.

The concept of regularity is introduced into CSG by considering its set operations to be the regularity-preserving variants of the usual point-set operations as defined below.

Definition 5.1 *The* regularized set operations *union*, intersection*, and set difference*, denoted by* $\cup^*$, $\cap^*$ *and* $\backslash^*$ *are defined as*

$$\begin{aligned} A \cup^* B &= c(i(A \cup B)) \\ A \cap^* B &= c(i(A \cap B)) \\ A \backslash^* B &= c(i(A \backslash B)) \end{aligned}$$

where $\cup$, $\cap$, *and* $\backslash$ *denote the usual set operations.*

If CSG primitives are chosen to be bounded regular sets, regularized set operations have the desirable property of being *algebraically closed* in the class of bounded regular sets. That is, every CSG tree is guaranteed to define a bounded regular set.

5.2.2 Algorithms for CSG Models

The CSG tree can be viewed as an implicit description of the geometry of the solid modeled that must be "evaluated" in some fashion in order to create graphical output or perform calculations.

The very structure of the CSG tree suggests the use of the very powerful and general "divide-and-conquer" approach to the design of algorithms for CSG trees. The basic idea of divide and conquer is to divide the problem in some fashion into two parts, recursively solve each part, and join the partial solutions to get the total solution. The recursion terminates when the problem has been subdivided into its "primitive" parts that allow a direct solution.

When applied to CSG trees, the natural way to subdivide a problem is to process the two subtrees of each interior node denoting a Boolean set operation of the tree separately. The recursion ends at leaves of the tree, where the problem is solved for one primitive. Solutions of subproblems are combined while taking the set operation at the node into respect.

Divide and conquer leads to efficient algorithms if the combination of subsolutions is simple and the problem can always be split into parts of approximately same size. Unfortunately, CSG trees are usually far from being balanced. Program 5.1 gives a generic divide and conquer algorithm for processing CSG trees. The following sections give specific instances of this schema.

Set Membership Classification

So-called *set membership classification* [121] algorithms are a particularly useful class of algorithms based on the divide and conquer approach. In general, a set membership classification algorithm works on two point sets, namely the *candidate set* C and the *reference set* R. The algorithm is expected to *classify* C against R by forming three sets $CinR$, $ConR$, and $CoutR$ representing the parts of C inside, on the boundary, and outside of R.

Specification of the "meaning" and the representations of the sets involved (C, R, $CinR$, $ConR$, $CoutR$) defines a particular set membership classification algorithm. For instance, the algorithm for classifying a finite line segment ("edge") against a CSG tree is cast into the general schema by choosing the sets as follows:

- C: An edge E given in terms of an ordered pair of coordinate triples;
- R: A CSG tree S;
- $CinR$, $ConR$, $CoutR$: Sets $EinS$, $EonS$, and $EoutS$, each being a set of nonempty subsegments of E such that $EinS \cup EonS \cup EoutS = E$, and their elements are inside, on the boundary, or outside S, respectively.

```
/* evaluate property P of a CSG tree */
P *Tree_P(S, args)
CSG_Tree *S;
{
        if(S->op == <primitive>)
                return(Primitive_P(S, args));
        else    return(Combine_P(Tree_P(S->Left, args),
                                 Tree_P(S->Right, args),
                                 S->Op));
}

/* evaluate P for a primitive */
P *Primitive_P(S, args)
{
        ...
}

/* combine two evaluations of P with set operation Op */
P *Combine_P(Left_P, Right_P, Op)
{
        ...
}
```

Program 5.1 Divide and conquer for CSG trees.


```
/* set membership classification of an edge vs. a CSG tree */
M *Tree_M(S, E)
CSG_Tree *S;
Edge *E;
{
        if(S->Op == <primitive>)
                return(Prim_M(S, E));
        else    return(Combine_M(Tree_M(S->Left, E),
                                 Tree_M(S->Right, E),
                                 S->Op));
}

/* classify E against a primitive */
M *Prim_M(S, E)
{
        ...
}

/* combine two classifications of E */
M *Combine_M(Left_M, Right_M, Op)
{
        ...
}
```

Program 5.2 Edge-solid classification.

The resulting algorithm of Program 5.2 nicely fits in the general approach of Program 5.1. The "property" evaluated is now the triple $M = \langle EinS, EonS, EoutS \rangle$. All we need to supply is an algorithm for classifying an edge against a primitive (Prim_M in Program 5.2), and an algorithm for combining two classifications (Combine_M).

Classification Against Primitive To do its task properly, the procedure Prim_M must calculate the intersections (if any) between E and the primitive, and subdivide E accordingly. For instance, in the case of Figure 5.5 the desired result of the primitive classification is the triple

$$
\begin{aligned}
M = <&\{(p_2, p_3)\}, && (EinS)\\
&\{\}, && (EonS)\\
&\{(p_1, p_2), (p_3, p_4)\} >. && (EoutS)
\end{aligned}
$$

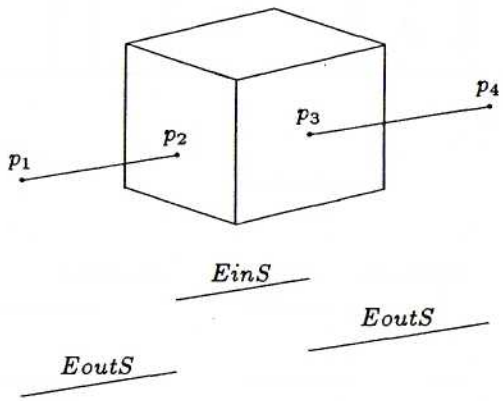

Figure 5.5 Edge vs. block primitive.

As all CSG primitives actually are combinations of half-spaces, the basic task needed is the classification of E against a half-space. This is a relatively straightforward operation.

For instance, suppose that we need to classify an edge E against a cylinder half-space. By transforming E appropriately, we can arrange things so that the cylinder is of the form

$$x^2 + y^2 - r^2 = 0,$$

and that E is represented by the parametric equation

$$E(t) = p_1 + t(p_2 - p_1), \; t = [0, 1]$$

where $p_i = (x_i \; y_i \; z_i)$ are the end points of E, and t is the line parameter. This gives us three simultaneous equations

$$\begin{aligned} x^2 + y^2 &= r^2 \\ x &= x_1 + t(x_2 - x_1) \\ y &= y_1 + t(y_2 - y_1). \end{aligned}$$

By substituting the second and third equation into the first, we get a second-order equation in t. If the equation has no real solutions, E does not intersect the cylinder at all; if a double root occurs, E is tangent to the cylinder; otherwise, the parameter values of two intersection points are given. What remains is to check whether the intersections occur within the

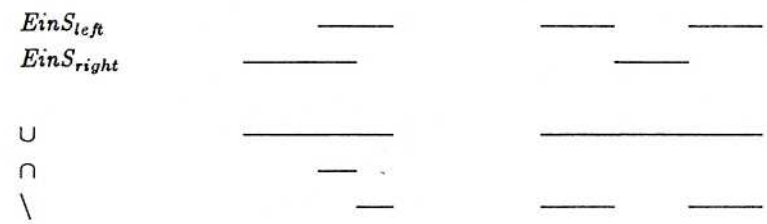

Figure 5.6 Combination rules.

edge, i.e., whether the parameter values are within 0 and 1. In the most complicated case of tori, a similar procedure yields a fourth-order polynomial in t, i.e., still something that can be solved with standard techniques.

Combination of Classifications Each of the results of edge-primitive classification ($EinS$, $EonS$, $EoutS$) consists of a sequence of edge segments. By arranging these sequences so that the segments appear in sorted order along E, the combination of classifications can be reduced to the merging of sets of line segments. The primitive task of this operation is the combination of just two edge segments by the Boolean set operation Op. For $EinS$ and $EoutS$ this leads to the combination rules illustrated in Figure 5.6.

Unfortunately, the rules for combining segments of the "left" $EonS$ with the "right" $EonS$ are not quite so simple. As illustrated in Figure 5.7, the combination rules for this "on-on"-case must take also the orientations of the respective surfaces into account.

Wire Frame Generation As an example on the use of the edge-solid classification, let us consider the *wire frame problem*:

> *Given a CSG tree S, determine a set* WireFrame *of line segments that form the "wire-frame" figure of S.*

Based on the edge-solid classification, the wire frame generation algorithm can be combined from the following steps:

1. *Generate*: Generate the set of all "tentative" edges by calculating the set of pairwise intersection curves of all half-spaces appearing in S.

2. *Classify*: Classify each tentative edge against S, and append the component $EonS$ to the result.

```
WireFrameGen(S)
CSGTree S;
{
    /* Generation step: */
    Tentative_list = empty;
    for each half-space G of S
    {
        for each half-space H of S
        {
            Tentative_list = union(Tentative_list, Intersect(G, H));
        }
    }

    /* Classification step: */
    WireFrame = empty;
    for each edge E of Tentative_list
    {
        <EinS, EonS, EoutS> = Tree_M(S, E);
        WireFrame = union(WireFrame, EonS);
    }
}
```

Program 5.3 Wire frame generation algorithm.

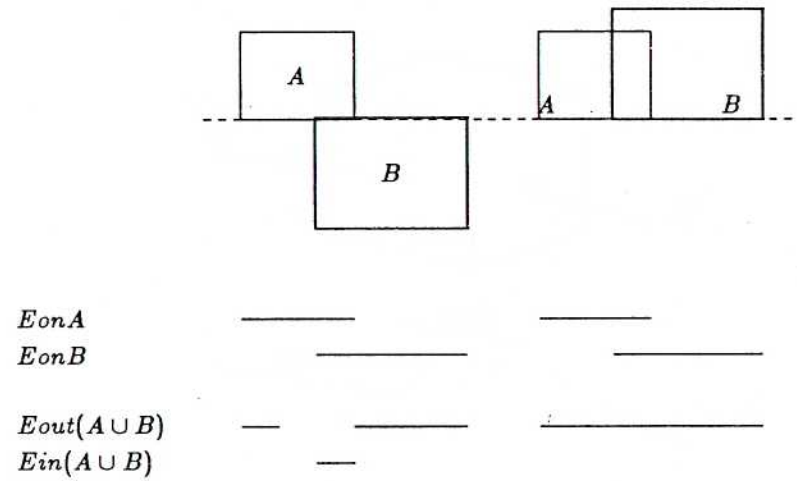

Figure 5.7 Combination rules for "on-on"-cases.

Program 5.3 outlines the resulting algorithm. The use of the complete three-way edge-solid classification may of course appear an overkill here, as only the component *EonS* really is needed. For more practical implementations of the algorithm, the reader is referred to Tilove [119].

Boundary Evaluation The generation of images with hidden lines removed requires a much more complicated algorithm that calculates the "faces" of the solid modeled via the CSG tree. This process is termed the *boundary evaluation.* Ordinarily, boundary evaluation is based on set membership classification between "primitive faces" (faces derived from CSG primitives) and CSG trees [96]. Of course, the capability of calculating intersections of quadratic surfaces is required [109].

In CSG modelers PADL-1 [126,97], PADL-2 [21], and GMSOLID [17] boundary evaluation is used to construct a complete boundary model based in the CSG tree. In particular, PADL-2 pursues so-called "incremental boundary evaluation" [119] that updates the boundary model so as to reflect changes in the CSG tree. These and other CSG modelers that can construct a boundary model from a CSG model can, of course, utilize algorithms for boundary models whenever they seem more appropriate than the self-contained methods for CSG models. Boundary models and their algorithms are discussed in Chapter 6.

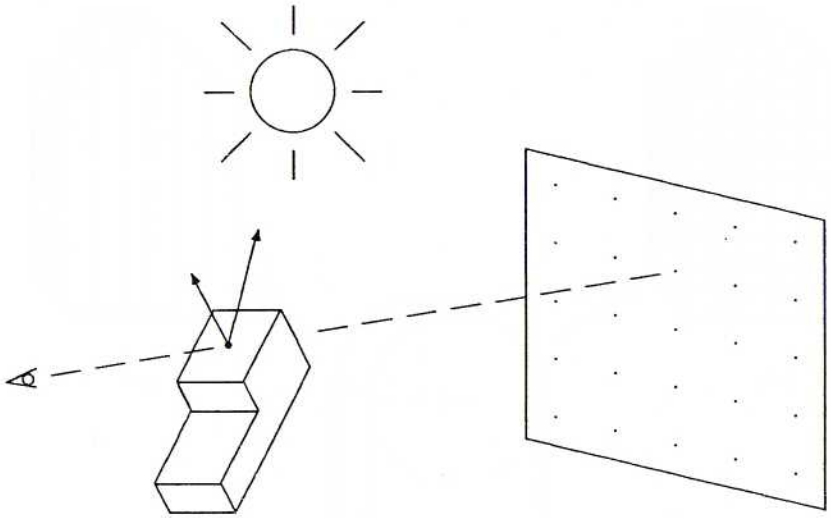

Figure 5.8 The ray casting paradigm.

Ray Casting

Another general approach to algorithm design for CSG models is the *ray casting* approach [104]. Ray casting is by its very nature approximative: instead of "evaluating" a CSG tree exactly, the tree is "sampled" along a finite number of lines, hence reducing the calculation of three-dimensional set operations to that of one-dimensional ones.

The term arises from the problem of generating an image of a CSG model for a color raster scan terminal (see Figure 5.8). In the ray casting paradigm, a "view ray" is sent from the location of the viewer through each pixel of the image. Based on the location where the ray first hits the object viewed, the shade and the intensity of the pixel are calculated. By sending secondary rays from the hit location, special visual effects such as shadowing, transparency, and mirroring can be created. This process is termed *ray tracing*.

The ray casting approach is actually a variation of the edge-solid classification, where instead of a finite edge, a semi-infinite ray is classified by a *ray classification* procedure. As a two-way "in-out" classification is sufficient, and as it can be assumed that the location of the "eye" is outside the solid, the ray classification procedure can represent the results by

means of a sequence of intersection points between the ray and the solid. An adequate representation is to have two arrays

$$\begin{array}{ccccc} t(1) & t(2) & t(3) & t(4) & \ldots \\ s(1) & s(2) & s(3) & s(4) & \ldots \end{array}$$

where $t(i)$'s give the parameter values of intersection points in a sorted order, and $s(i)$'s identify the half-spaces intersected at these points. The ray segments $[0, t(1)]$, $[t(2), t(3)]$, and so on are considered to occur outside the solid, and segments $[t(1), t(2)]$, $[t(3), t(4)]$ inside of it.

The implementation of the ray-classification procedure follows similar lines as the edge-solid classification, being only simpler. For clarity, the outline of the algorithm given in Program 5.4 indicates the intersection tests needed between the ray and the half-spaces of the primitives.

The image generation algorithm can now simply pick the first intersection of the sequence, and calculate the color based on the information it has on normal vector and color of the surface intersected and the locations of light sources.

Ray casting is useful not only for color image generation, but also for calculating line drawings and integral properties of solids; see, e.g., [44]. The problem is that the amount of computation needed is large: to calculate an image at 1000^2 resolution, one million ray classifications are needed. As each classification is a recursive algorithm potentially involving numerical solutions of fourth order polynomials, the speed of ray casting algorithms is very far from interactive.

Integral Properties

The approximative evaluation of integral properties (such as volume) of CSG models is ordinarily based on conversion algorithms that (implicitly) construct a decomposition model approximating the solid. The evaluation of the property then reduces to calculating the contribution of each cell of the decomposition to the total result.

The conversion algorithms are generally based on the set membership classification and ray casting approaches. Several alternative arrangements of cells are possible, each corresponding to a particular set membership classification algorithm. Various alternatives are shown in Figure 5.9 [98,69,68].

For instance, the case of "column decomposition" corresponds with ordinary ray casting. A column is determined by sending a ray along its center; where the ray is inside the solid, a solid column is created. This algorithm is used in solid modelers employing the ray casting approach, e.g., in the SYNTHAVISION™ solid modeler [45].

```
Classification
RayCast(S, R)
CSGTree *S;
Ray *R;
{
    if(S->Op == <set operation>)
    {
        LeftClassification = RayCast(S->Left, R);
        RightClassification = RayCast(S->Right, R);
        return(Combine(LeftClassification,
                       RightClassification,
                       S->Op);
    }
    else
    {
        switch(S->Op)
        {
            case "block":
                do 6 ray-plane intersection tests
            case "sphere":
                do 1 ray-quadratic intersection test
            case "cylinder":
                do 2 ray-plane and 1 ray-quadratic tests
            case "cone":
                do 1 ray-plane and 1 ray-quadratic test
            case "torus":
                do 1 ray-quartic intersection test
        }
        Classification = results of tests;
        return(Classification);
    }
}
```

Program 5.4 Ray classification algorithm.

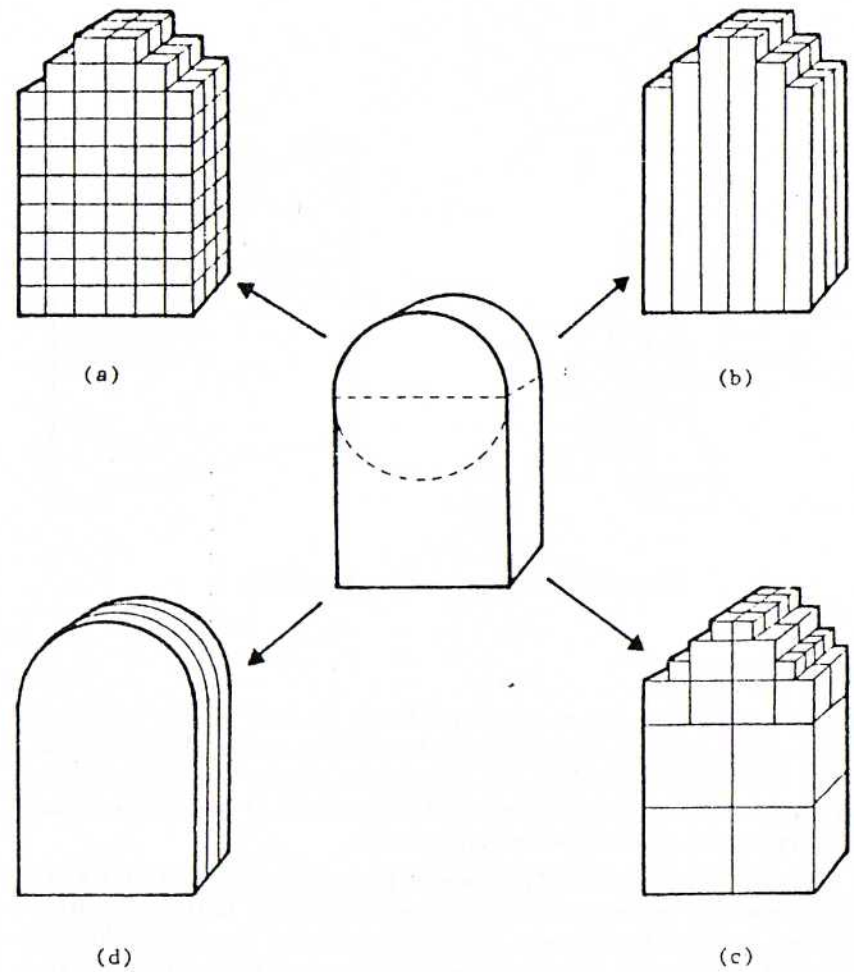

Figure 5.9 CSG-to-cell conversions.

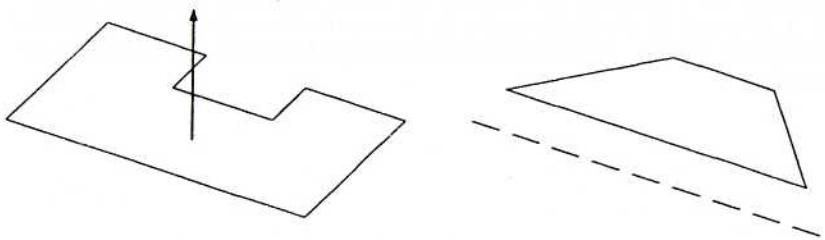

Figure 5.10 Sweeping primitives.

The simplest case, exhaustive enumeration, is based on *point-solid classification*: given a point P (the center point of the cell), determine whether P is inside, on the boundary, or outside the solid. CSG is directly useful for answering this kind of a question as classification with respect to primitives amounts to substituting P to the defining function of each half-space. For the two-way classification, the combination rules are simple and left to the reader. This method is used in TIPS.

5.2.3 Generalized CSG Primitives

To apply the set membership classification and ray casting approaches to CSG algorithms for a particular CSG model, the essential task reduces to writing procedures that perform the underlying operations of these algorithms. Correspondingly, *any* primitive for which such procedures can be written can potentially be included into a CSG modeler. In particular, there is no need to restrict CSG primitives to be simple collections of half-spaces.

Many CSG-based systems follow this approach, and allow the inclusion of *generalized CSG primitives* of various types into their CSG models. For instance, *polyhedral primitives* can be added into a ray casting modeler by offering sufficient facilities for their creation, and a ray casting algorithm for arbitrary polyhedra. This approach leads to a hybrid of the CSG and boundary modeling approaches.

Another potentially useful addition is the inclusion of *sweeping primitives*, modeling primitives of the *sweeping model* approach to solid modeling. A *translational sweeping primitive* is defined by a *base set*, a bounded subset of a plane, and a *sweeping direction*; informally, the primitive consists of the set of points "swept" by the base set as it moves along the sweeping direction. A *rotational sweeping primitive* is similarly defined by a base set and a rotation axis, and consists of the set of points swept by the

base set as it rotates around the axis. Of course, restrictions on admissible base sets must be posed so as to guarantee that the resulting objects are valid solids.

In this context, the essential fact is that there are algorithms, for example, for the ray casting of translational and rotational sweeping primitives directly (without conversion to some other representation); see [62,125,124]. Hence they can be included into the domain of a ray casting modeler. Moreover, "two-and-half-dimensional" models generated by sweeping simple outlines consisting of straight lines and arcs along a normal axis can be converted directly to CSG models; see [128].

5.2.4 Properties of CSG Models

The properties of CSG models can be summarized similarly as those of other models we have discussed.

Expressive power: Depends on the class of half-spaces available. Cannot easily be extended to cover surface patches.

Validity: Every CSG tree is guaranteed to model a valid solid object, provided that the primitives are valid (i.e., bounded regular sets).

Unambiguity and uniqueness: Every CSG tree unambiguously models a solid. They are not unique.

Description languages: Usually textual languages. It is possible to include a graphical interface into a CSG modeler.

Conciseness: CSG trees are in principle relatively concise. In practical modelers, they tend to grow as other information aiming at efficient graphical operations is attached to the basic CSG tree.

Closure of operations: Set operations are algebraically closed for CSG trees.

Computational ease and applicability: The computational power some important CSG algorithms (such as boundary evaluation) is poor. However, as their basic steps are very simple, they are prime candidates for VLSI implementation. In cases such as the point classification for conversion to a decomposition model, the divide and conquer approach generally leads to efficient algorithms, although their efficiency deteriorates if the CSG tree is ill-balanced.

CSG models are based on rigorous mathematical background, and the families of algorithms available for them are well understood and applicable to many problems. Because of these reasons, many commercial CSG modelers are available and in wide use.

PROBLEMS

5.1. Give a Boolean expression of half-spaces for each of the CSG primitives of Figure 5.3.

5.2. Specify and write a ray-sphere intersection procedure based on the general outline of Program 5.4. In addition to the ray parameter value of the intersection point, the procedure should also calculate the surface normal of the sphere at the intersection point for use of the ray tracing program of Problem 5.3.

5.3. Based on the ray-sphere intersection procedure of Problem 5.2, specify and write a ray tracing program that can display images (possibly intersecting) spheres on a color raster display.

Hint: You will need to apply a suitable shading method based on the surface normal of the intersection of the ray with a sphere and the location(s) of the light source(s). See, e.g., [39,88] for additional information.

5.4. Generalize your ray tracing program of Problem 5.3 by including additional primitive types, such as blocks, cylinders, and cones. Try not be become too hooked on pretty pictures!

BIBLIOGRAPHIC NOTES

The basic ideas of constructive modeling can be traced back to late 60's and early 70's. Ray casting was probably introduced by Appel [5]. In this context, many aspects of CSG modeling were quite early developed. Probably the first practical modeler was the original SYNTHAVISION system [46].

As a general technique for CSG modeling, ray casting was introduced by Roth [104].

The rigorous theory of CSG modeling was developed by the workers of the PADL-project at the University of Rochester [126,97,94]. The project constructed two successive CSG modelers, PADL-1 and PADL-2, which became widely known in the industry and the academia. They were eventually used as a basis for commercial development.

Advanced algorithmic techniques for CSG diminish the amount of processing by using several techniques. The thesis of Tilove [119] is a basic reference to many of them. A good example of these techniques is the *null object detection* algorithm described in [120]. A null object detection algorithm is expected to test whether a given CSG tree models the empty set or not.

The *pruning* of a CSG tree tries to diminish the amount of processing by modifying a CSG tree into a smaller equivalent tree. Woodwark uses a spatial subdivision and pruning technique to speed up the generation of images from a CSG representation [133,132]. Samet and Tamminen [107] use a similar technique for the conversion of CSG models to bintrees.

Figure 5.9, page 95 has been previously published as Figure 1 of the article "Algorithms for computing the volume and other integral properties of solids" by Y. T. Lee and A. A. G. Requicha in *Communications of the ACM*, Volume 25, Number 9. ©1982, Association for Computing Machinery, Inc. Reprinted by permission.